Generics in C#

A Comprehensive Guide

From Historical Context to Modern Patterns

Part I: Historical Context

Chapter 1: The World Before Generics

1.1 The Problem of Type Safety

Before generics were introduced, developers faced a fundamental tension in programming: **flexibility versus type safety**.

Imagine you're building a collection to store items. You want it to be reusable—capable of storing integers today, strings tomorrow, and custom Employee objects next week. In the early days of C# (versions 1.0 and 1.1), you had two imperfect options.

Option A: Create Type-Specific Collections

```
public class IntegerList
{
    private int[] _items;
    public void Add(int item) { /* ... */ }
    public int Get(int index) { /* ... */ }
}

public class StringList
{
    private string[] _items;
    public void Add(string item) { /* ... */ }
    public string Get(int index) { /* ... */ }
}
```

This approach provided **type safety**—the compiler would catch errors like adding a string to an IntegerList. However, it was utterly **impractical**. For every type you needed to store, you had to write an entirely new collection class. Code duplication ran rampant.

Option B: Use System.Object

The .NET Framework offered a seemingly elegant solution: since every type in .NET inherits from System.Object, you could create a single collection that stores objects:

```
public class ArrayList
{
    private object[] _items;

    public void Add(object item) { /* ... */ }
    public object Get(int index) { /* ... */ }
}
```

This was the approach taken by System.Collections.ArrayList, and it worked—sort of. You could store anything. But this flexibility came at a severe cost.

1.2 The Two Penalties of Object-Based Collections

Penalty #1: Boxing and Unboxing (Performance)

In .NET, types are divided into two categories: **Value types** (e.g., int, double, bool, struct) stored directly on the stack, and **Reference types** (e.g., string, classes) stored on the heap with a reference on the stack.

When you store a value type in an object-based collection, the runtime must perform **boxing**: wrapping the value in an object on the heap. When retrieving, **unboxing** occurs.

```
ArrayList numbers = new ArrayList();
numbers.Add(42);  // Boxing occurs here: int → object
```

```
int value = (int)numbers[0];  // Unboxing: object → int
```

Boxing and unboxing are expensive operations involving memory allocation on the heap, copying data between stack and heap, and additional garbage collection pressure. In tight loops processing thousands of items, this overhead was devastating to performance.

Penalty #2: Loss of Type Safety (Correctness)

With object-based collections, the compiler had no way to enforce what types were stored:

```
ArrayList list = new ArrayList();
list.Add(1);
list.Add(2);
list.Add("three");  // Compiles fine!

foreach (object item in list)
{
    int number = (int)item;  // Runtime exception on "three"!
}
```

Errors that should have been caught at compile time now exploded at runtime—sometimes in production, sometimes in edge cases that testing missed. This violated a core principle of strongly-typed languages: **catch errors as early as possible**.

1.3 The Industry Response

The limitations of object-based collections were not unique to C#. The Java community faced identical challenges, and in 2004, Java 5.0 introduced generics. C++ had long offered templates, a compile-time mechanism for generic programming.

Microsoft and the C# team, led by Anders Hejlsberg, recognized that a robust generics implementation was essential. However, they made a crucial design decision that differentiated C# generics from Java's approach.

Part II: The Advent of Generics

Chapter 2: Generics Arrive in C# 2.0

In November 2005, C# 2.0 was released alongside .NET Framework 2.0. Generics were the flagship feature.

2.1 The Core Concept

A **generic** is a type or method that is parameterized by one or more type parameters. Instead of committing to a specific type, you define a "placeholder" that gets filled in when the generic is used.

```
public class List<T>
{
    private T[] _items;

    public void Add(T item) { /* ... */ }
    public T Get(int index) { /* ... */ }
}
```

The <T> is a **type parameter**. By convention: T for general purpose, TKey/TValue for key-value scenarios, TInput/TOutput for transformation scenarios, and TEntity for entity/model scenarios.

When you use the generic, you provide a **type argument**:

```
List<int> numbers = new List<int>();
List<string> names = new List<string>();
List<Employee> employees = new List<Employee>();
```

Each of these is a **constructed type**—a specific version of the generic with concrete types substituted for the parameters.

2.2 The Benefits Realized

Benefit #1: Type Safety Restored

The compiler now enforces type correctness:

```
List<int> numbers = new List<int>();
numbers.Add(1);
numbers.Add(2);
numbers.Add("three");  // Compile error! Cannot convert string to int
```

Errors are caught immediately, in your IDE, before the code ever runs.

Benefit #2: No Boxing for Value Types

The .NET runtime generates specialized versions of generic types for value types. When you use List<int>, the runtime creates a version that works directly with integers—no boxing required.

```
List<int> numbers = new List<int>();
numbers.Add(42);  // No boxing! Stored directly as int
int value = numbers[0];  // No unboxing! Retrieved directly as int
```

Benefit #3: Code Reuse Without Sacrifice

A single generic class serves infinite use cases:

```
// One implementation, infinite applications
List<int> integers = new List<int>();
List<string> strings = new List<string>();
List<Customer> customers = new List<Customer>();
List<List<int>> matrix = new List<List<int>>();  // Generics can nest!
```

2.3 Reification: C#'s Secret Weapon

Here lies a critical distinction between C# and Java generics.

Java uses type erasure: generic type information exists only at compile time. At runtime, a List<String> and a List<Integer> are both just List. This was done for backward compatibility with older JVMs but limits what generics can do.

C# uses reification: generic type information is preserved at runtime. The CLR (Common Language Runtime) knows the difference between List<int> and List<string>. This enables:

```
// You can check generic types at runtime
Type listType = typeof(List<int>);
Console.WriteLine(listType.IsGenericType);  // True
Console.WriteLine(listType.GetGenericArguments()[0]);  // System.Int32

// You can create generic types dynamically
Type openType = typeof(List<>);
Type closedType = openType.MakeGenericType(typeof(string));
object instance = Activator.CreateInstance(closedType);
```

This design decision makes C# generics more powerful and enables sophisticated scenarios in frameworks like Entity Framework, dependency injection containers, and serialization libraries.

Part III: Mastering Generic Syntax

Chapter 3: Generic Classes and Structures

3.1 Defining a Generic Class

Let's build a practical example—a generic Result<T> type for handling operation outcomes:

```
public class Result<T>
{
    public bool IsSuccess { get; }
    public T Value { get; }
    public string Error { get; }

    private Result(bool isSuccess, T value, string error)
    {
        IsSuccess = isSuccess;
        Value = value;
        Error = error;
    }

    public static Result<T> Success(T value)
    {
        return new Result<T>(true, value, null);
    }

    public static Result<T> Failure(string error)
    {
        return new Result<T>(false, default, error);
    }
}
```

Usage:

```
public Result<Employee> GetEmployee(int id)
{
    var employee = _database.Find(id);

    if (employee == null)
        return Result<Employee>.Failure("Employee not found");

    return Result<Employee>.Success(employee);
}

// Calling code
var result = GetEmployee(42);
if (result.IsSuccess)
    Console.WriteLine(result.Value.Name);
else
    Console.WriteLine(result.Error);
```

3.2 Multiple Type Parameters

Generics can have multiple type parameters:

```
public class Pair<TFirst, TSecond>
{
    public TFirst First { get; set; }
    public TSecond Second { get; set; }

    public Pair(TFirst first, TSecond second)
    {
        First = first;
        Second = second;
    }
```

```
}

// Usage
var pair = new Pair<string, int>("Age", 30);
var coordinate = new Pair<double, double>(45.5, 122.7);
```

3.3 Generic Structs

Value types can also be generic:

```
public struct Point<T>
{
    public T X { get; }
    public T Y { get; }

    public Point(T x, T y)
    {
        X = x;
        Y = y;
    }
}

// Usage
Point<int> pixelPoint = new Point<int>(100, 200);
Point<double> precisePoint = new Point<double>(3.14159, 2.71828);
```

Chapter 4: Generic Methods

4.1 Methods with Type Parameters

Methods can be generic independently of their containing class:

```
public class Utilities
{
    public static void Swap<T>(ref T a, ref T b)
    {
        T temp = a;
        a = b;
        b = temp;
    }
}

// Usage
int x = 5, y = 10;
Utilities.Swap<int>(ref x, ref y);  // x = 10, y = 5

string first = "Hello", second = "World";
Utilities.Swap<string>(ref first, ref second);  // Swapped!
```

4.2 Type Inference

The compiler can often infer type arguments:

```
int x = 5, y = 10;
Utilities.Swap(ref x, ref y);  // Compiler infers <int>

string a = "Hello", b = "World";
Utilities.Swap(ref a, ref b);  // Compiler infers <string>
```

This makes generic methods feel as natural as non-generic ones.

4.3 Generic Methods in Non-Generic Classes

A common pattern in utility classes:

```
public static class CollectionExtensions
{
    public static bool IsEmpty<T>(this ICollection<T> collection)
    {
        return collection == null || collection.Count == 0;
    }

    public static T FirstOrDefault<T>(this IEnumerable<T> source, T defaultValue)
    {
        foreach (T item in source)
            return item;
        return defaultValue;
    }
}

// Usage
List<int> numbers = new List<int>();
bool empty = numbers.IsEmpty();  // True
int first = numbers.FirstOrDefault(-1);  // Returns -1
```

Chapter 5: Generic Interfaces

5.1 Defining Generic Interfaces

Interfaces can be parameterized just like classes:

```
public interface IRepository<TEntity> where TEntity : class
{
    TEntity GetById(int id);
    IEnumerable<TEntity> GetAll();
    void Add(TEntity entity);
    void Update(TEntity entity);
    void Delete(TEntity entity);
}
```

5.2 Implementing Generic Interfaces

```
public class EmployeeRepository : IRepository<Employee>
{
    private readonly AppDbContext _context;

    public EmployeeRepository(AppDbContext context)
    {
        _context = context;
    }

    public Employee GetById(int id) => _context.Employees.Find(id);
    public IEnumerable<Employee> GetAll() => _context.Employees.ToList();
    public void Add(Employee entity) => _context.Employees.Add(entity);
    public void Update(Employee entity) => _context.Employees.Update(entity);
    public void Delete(Employee entity) => _context.Employees.Remove(entity);
}
```

5.3 Key Generic Interfaces in .NET

The .NET Base Class Library is rich with generic interfaces:

Interface	Purpose
IEnumerable<T>	Iteration over a sequence
ICollection<T>	Sized, modifiable collection
IList<T>	Indexed access
IDictionary<TKey, TValue>	Key-value mapping
IComparable<T>	Type-safe comparison
IEquatable<T>	Type-safe equality
IComparer<T>	External comparison logic
IEqualityComparer<T>	External equality logic

Part IV: Constraints — Controlling Generic Flexibility

Chapter 6: The Need for Constraints

6.1 The Problem: Too Much Freedom

Consider this innocent-looking generic method:

```
public static T Max<T>(T a, T b)
{
    if (a > b)  // Compile error!
        return a;
    return b;
}
```

This won't compile. Why? The compiler knows nothing about T. It could be int, which supports >, or it could be Employee, which might not. Without knowing what operations T supports, the compiler must reject the code.

6.2 Constraints to the Rescue

Constraints tell the compiler: "I promise T will have certain capabilities."

```
public static T Max<T>(T a, T b) where T : IComparable<T>
{
    if (a.CompareTo(b) > 0)
        return a;
    return b;
}
```

Now the compiler knows T implements IComparable<T>, which guarantees the CompareTo method exists.

Chapter 7: Types of Constraints

7.1 Reference Type Constraint (class)

Restricts T to reference types (classes, interfaces, delegates, arrays):

```
public class EntityCache<T> where T : class
{
    private T _cached;

    public T GetOrDefault()
    {
        return _cached;  // Can be null
    }
}
```

7.2 Value Type Constraint (struct)

Restricts T to non-nullable value types:

```
public struct Nullable<T> where T : struct
{
    private readonly T _value;
    private readonly bool _hasValue;

    // This is essentially how System.Nullable<T> works
}
```

7.3 Constructor Constraint (new())

Requires T to have a public parameterless constructor:

```
public static T CreateInstance<T>() where T : new()
{
    return new T();  // Now this is valid!
}

// Usage
var employee = CreateInstance<Employee>();  // Works if Employee has parameterless ctor
```

7.4 Base Class Constraint

Restricts T to a specific class or its descendants:

```
public class AnimalShelter<T> where T : Animal
{
    private List<T> _animals = new List<T>();

    public void Admit(T animal)
    {
        animal.Feed();  // Valid: Animal has Feed() method
        _animals.Add(animal);
    }
}

// Usage
var dogShelter = new AnimalShelter<Dog>();  // Dog : Animal
var catShelter = new AnimalShelter<Cat>();  // Cat : Animal
// var carShelter = new AnimalShelter<Car>();  // Error!
```

7.5 Interface Constraint

Restricts T to types implementing specific interfaces:

```
public class SortedCollection<T> where T : IComparable<T>
{
    private List<T> _items = new List<T>();

    public void Add(T item)
    {
        int index = _items.BinarySearch(item);  // Uses IComparable<T>
        if (index < 0) index = ~index;
        _items.Insert(index, item);
    }
}
```

7.6 Multiple Constraints

Constraints can be combined:

```
public class Repository<TEntity>
    where TEntity : class, IEntity, new()
{
    public TEntity Create()
    {
        var entity = new TEntity();  // new() constraint
        entity.Id = GenerateId();    // IEntity constraint
        return entity;               // class constraint allows null checks
    }
}
```

7.7 The notnull Constraint (C# 8.0+)

In nullable reference type contexts, ensures T cannot be null:

```
public class NonNullCache<T> where T : notnull
{
    private Dictionary<string, T> _cache = new();

    public void Set(string key, T value)
    {
        _cache[key] = value;  // value is guaranteed non-null
    }
}
```

7.8 The unmanaged Constraint (C# 7.3+)

Restricts to unmanaged types (primitives and structs containing only unmanaged types):

```
public unsafe void ProcessBuffer<T>(T* buffer, int length)
    where T : unmanaged
{
    // Can use pointer arithmetic with T
    for (int i = 0; i < length; i++)
    {
        Process(buffer[i]);
    }
}
```

Chapter 8: Constraint Combinations and Rules

8.1 Constraint Ordering

When specifying multiple constraints, they must appear in a specific order:

- 1. class or struct (mutually exclusive)
- 2. Base class name (if any)
- 3. Interface names
- 4. new() (must be last)

```
// Correct order
public class Example<T>
    where T : class, BaseClass, IInterface1, IInterface2, new()
{ }
```

8.2 Constraints on Multiple Type Parameters

Each type parameter can have its own constraints:

```
public class Mapper<TSource, TDestination>
    where TSource : class
    where TDestination : class, new()
{
    public TDestination Map(TSource source)
    {
        var destination = new TDestination();
        // Map properties...
        return destination;
    }
}
```

Part V: Advanced Topics

Chapter 9: Covariance and Contravariance

9.1 The Variance Problem

Consider this scenario:

```
class Animal { }
class Dog : Animal { }

List<Dog> dogs = new List<Dog>();
List<Animal> animals = dogs;  // Compile error! Why?
```

Intuitively, since every Dog is an Animal, a list of dogs should be usable as a list of animals. But consider: animals.Add(new Cat()); — This would corrupt the dog list! This is why List<T> doesn't allow such assignments. But sometimes variance is safe...

9.2 Covariance (out keyword)

Covariance allows a generic type to preserve the inheritance relationship of its type argument. It's safe when the type parameter is only used in **output** positions:

```
public interface IProducer<out T>
{
    T Produce();  // T only appears as output (return type)
}

class DogProducer : IProducer<Dog>
{
    public Dog Produce() => new Dog();
}

// Now this is valid:
IProducer<Animal> producer = new DogProducer();
Animal animal = producer.Produce();  // Safe! We get a Dog, which is an Animal
```

IEnumerable<T> is covariant:

```
IEnumerable<Dog> dogs = new List<Dog>();
IEnumerable<Animal> animals = dogs;  // Valid! IEnumerable only produces items
```

9.3 Contravariance (in keyword)

Contravariance reverses the inheritance relationship. It's safe when the type parameter is only used in **input** positions:

```
public interface IConsumer<in T>
{
    void Consume(T item);  // T only appears as input (parameter)
}

class AnimalConsumer : IConsumer<Animal>
{
    public void Consume(Animal animal) => Console.WriteLine("Fed an animal");
}

// Now this is valid:
IConsumer<Dog> dogConsumer = new AnimalConsumer();
dogConsumer.Consume(new Dog());  // Safe! AnimalConsumer can handle any Animal
```

9.4 Invariance

When a type parameter appears in both input and output positions, variance is not possible:

```
public interface ITransformer<T>
{
    T Transform(T input);  // T is both input and output
}
// ITransformer<T> is invariant
```

Chapter 10: Static Members in Generic Types

10.1 Static Fields Are Per Constructed Type

A critical detail often overlooked:

```
public class Counter<T>
{
    public static int Count = 0;

    public Counter()
    {
        Count++;
    }
}

// Usage
var intCounter1 = new Counter<int>();
var intCounter2 = new Counter<int>();
var stringCounter = new Counter<string>();

Console.WriteLine(Counter<int>.Count);     // 2
Console.WriteLine(Counter<string>.Count);  // 1
```

Each constructed type (Counter<int>, Counter<string>) has its own static field. They are completely independent.

10.2 Static Constructors

Static constructors also run once per constructed type:

```
public class TypeLogger<T>
{
    static TypeLogger()
    {
        Console.WriteLine($"TypeLogger<{typeof(T).Name}> initialized");
    }
}

// Using TypeLogger<int> and TypeLogger<string> prints two messages
```

Chapter 11: Generic Type Inference

11.1 How Inference Works

The compiler examines method arguments to infer type parameters:

```
public static List<T> CreateList<T>(T first, T second)
{
    return new List<T> { first, second };
}

// Compiler infers T from arguments
var ints = CreateList(1, 2);           // List<int>
var strings = CreateList("a", "b");    // List<string>
```

11.2 Inference Limitations

Inference only works from arguments, not return types:

```
public static T Parse<T>(string value) { /* ... */ }

// This won't work:
int number = Parse("42");  // Error: Cannot infer T

// You must specify:
int number = Parse<int>("42");
```

11.3 Complex Inference

The compiler can infer through multiple levels:

```
public static TResult Select<TSource, TResult>(
    TSource source,
    Func<TSource, TResult> selector)
{
    return selector(source);
}

// Both TSource and TResult are inferred
string result = Select(42, n => n.ToString());
// TSource = int, TResult = string
```

Chapter 12: Reflection and Generics

12.1 Examining Generic Types at Runtime

```
Type listType = typeof(List<int>);

Console.WriteLine(listType.IsGenericType);           // True
Console.WriteLine(listType.IsConstructedGenericType); // True

Type[] args = listType.GetGenericArguments();
Console.WriteLine(args[0]);  // System.Int32

Type definition = listType.GetGenericTypeDefinition();
Console.WriteLine(definition);  // System.Collections.Generic.List`1[T]
```

12.2 Open vs. Closed Generic Types

```
Type openType = typeof(Dictionary<,>);   // Open: no type arguments
Type closedType = typeof(Dictionary<string, int>);  // Closed: fully specified

Console.WriteLine(openType.IsGenericTypeDefinition);    // True
Console.WriteLine(closedType.IsGenericTypeDefinition);  // False
```

12.3 Creating Generic Types Dynamically

```
Type openList = typeof(List<>);
Type closedList = openList.MakeGenericType(typeof(string));
object instance = Activator.CreateInstance(closedList);

// instance is now a List<string>
var list = (List<string>)instance;
list.Add("Dynamic!");
```

12.4 Invoking Generic Methods via Reflection

```
public class Processor
{
    public void Process<T>(T item)
    {
        Console.WriteLine($"Processing {typeof(T).Name}: {item}");
    }
}

// Invoke via reflection
var processor = new Processor();
MethodInfo openMethod = typeof(Processor).GetMethod("Process");
MethodInfo closedMethod = openMethod.MakeGenericMethod(typeof(int));
closedMethod.Invoke(processor, new object[] { 42 });
```

Part VI: Patterns and Best Practices

Chapter 13: Common Generic Patterns

13.1 The Repository Pattern

```
public interface IRepository<TEntity> where TEntity : class, IEntity
{
    Task<TEntity?> GetByIdAsync(int id);
    Task<IReadOnlyList<TEntity>> GetAllAsync();
    Task<TEntity> AddAsync(TEntity entity);
    Task UpdateAsync(TEntity entity);
    Task DeleteAsync(TEntity entity);
}

public class Repository<TEntity> : IRepository<TEntity>
    where TEntity : class, IEntity
{
    protected readonly DbContext _context;
    protected readonly DbSet<TEntity> _dbSet;

    public Repository(DbContext context)
    {
        _context = context;
        _dbSet = context.Set<TEntity>();
    }

    public virtual async Task<TEntity?> GetByIdAsync(int id)
    {
        return await _dbSet.FindAsync(id);
    }

    // ... other implementations
}
```

13.2 The Factory Pattern

```
public interface IFactory<out T>
{
    T Create();
}

public class ServiceFactory<T> : IFactory<T> where T : class, new()
{
    public T Create() => new T();
}

// With parameters using Func<T>
public class ConfigurableFactory<T>
{
    private readonly Func<T> _creator;

    public ConfigurableFactory(Func<T> creator)
    {
        _creator = creator;
    }

    public T Create() => _creator();
}
```

13.3 The Specification Pattern

```
public interface ISpecification<T>
{
```

```
    bool IsSatisfiedBy(T entity);
    Expression<Func<T, bool>> ToExpression();
}

public abstract class Specification<T> : ISpecification<T>
{
    public bool IsSatisfiedBy(T entity)
    {
        return ToExpression().Compile()(entity);
    }

    public abstract Expression<Func<T, bool>> ToExpression();

    public Specification<T> And(Specification<T> other)
    {
        return new AndSpecification<T>(this, other);
    }
}

// Usage
public class ActiveEmployeeSpecification : Specification<Employee>
{
    public override Expression<Func<Employee, bool>> ToExpression()
    {
        return e => e.IsActive && e.TerminationDate == null;
    }
}
```

13.4 The Result/Either Pattern

```
public class Result<TSuccess, TFailure>
{
    public TSuccess? Success { get; }
    public TFailure? Failure { get; }
    public bool IsSuccess { get; }

    public static Result<TSuccess, TFailure> Ok(TSuccess value) => new(value);
    public static Result<TSuccess, TFailure> Fail(TFailure error) => new(error);

    public TResult Match<TResult>(
        Func<TSuccess, TResult> onSuccess,
        Func<TFailure, TResult> onFailure)
    {
        return IsSuccess ? onSuccess(Success!) : onFailure(Failure!);
    }
}

// Usage
var result = CreateEmployee("Alice", "alice@example.com");
var message = result.Match(
    onSuccess: emp => $"Created employee: {emp.Name}",
    onFailure: err => $"Error: {err}"
);
```

Chapter 14: Best Practices

14.1 Naming Conventions

- Single type parameter: T
- Multiple parameters: Descriptive names starting with T

```
// Good
Dictionary<TKey, TValue>
Func<TInput, TOutput>
IRepository<TEntity>

// Avoid
Dictionary<K, V>  // Not immediately clear
List<Type>        // Confusing with System.Type
```

14.2 Prefer Generic Collections

Always prefer System.Collections.Generic over System.Collections:

```
// Good
List<Employee> employees = new();
Dictionary<string, int> scores = new();

// Avoid
ArrayList employees = new();  // Boxing, no type safety
Hashtable scores = new();     // Boxing, no type safety
```

14.3 Use Constraints Judiciously

Add constraints only when necessary:

```
// Over-constrained: Do we really need all these?
public class Repository<T>
    where T : class, IEntity, IValidatable, IAuditable, new()

// Better: Only what's actually used
public class Repository<T> where T : class, IEntity
```

14.4 Consider Variance When Designing Interfaces

If your interface only produces T, make it covariant:

```
public interface IReadOnlyRepository<out TEntity>
{
    TEntity GetById(int id);
    IEnumerable<TEntity> GetAll();
}
```

If it only consumes T, make it contravariant:

```
public interface IValidator<in T>
{
    bool Validate(T item);
}
```

14.5 Avoid Excessive Generic Nesting

```
// Hard to read
Dictionary<string, List<Tuple<int, Dictionary<string, object>>>> data;
```

```
// Better: Create meaningful types
class UserData { /* ... */ }
Dictionary<string, UserData> data;
```

Part VII: Generics in the .NET Ecosystem

Chapter 15: Generics in Entity Framework Core

15.1 DbSet<T>

```
public class AppDbContext : DbContext
{
    public DbSet<Employee> Employees { get; set; }
    public DbSet<Department> Departments { get; set; }
}

// DbSet<T> provides generic query and change tracking
var employees = await context.Employees
    .Where(e => e.IsActive)
    .ToListAsync();
```

15.2 Generic Configuration

```
public abstract class BaseEntityConfiguration<T> : IEntityTypeConfiguration<T>
    where T : BaseEntity
{
    public virtual void Configure(EntityTypeBuilder<T> builder)
    {
        builder.HasKey(e => e.Id);
        builder.Property(e => e.CreatedAt).IsRequired();
        builder.Property(e => e.UpdatedAt).IsRequired();
    }
}

public class EmployeeConfiguration : BaseEntityConfiguration<Employee>
{
    public override void Configure(EntityTypeBuilder<Employee> builder)
    {
        base.Configure(builder);
        builder.Property(e => e.Name).HasMaxLength(100);
    }
}
```

Chapter 16: Generics in Dependency Injection

16.1 Generic Service Registration

```
// Register open generic
services.AddScoped(typeof(IRepository<>), typeof(Repository<>));

// Now any IRepository<T> resolves to Repository<T>
// IRepository<Employee> → Repository<Employee>
// IRepository<Product> → Repository<Product>
```

16.2 Generic Decorators

```
public class CachingRepository<T> : IRepository<T> where T : class, IEntity
{
    private readonly IRepository<T> _inner;
    private readonly IMemoryCache _cache;

    public CachingRepository(IRepository<T> inner, IMemoryCache cache)
    {
        _inner = inner;
        _cache = cache;
    }

    public async Task<T?> GetByIdAsync(int id)
    {
        string key = $"{typeof(T).Name}_{id}";
        return await _cache.GetOrCreateAsync(key,
            entry => _inner.GetByIdAsync(id));
    }
}

// Registration with decoration (using Scrutor)
services.Decorate(typeof(IRepository<>), typeof(CachingRepository<>));
```

Chapter 17: Generics in Blazor

17.1 Generic Components

```
@typeparam TItem

<ul>
    @foreach (var item in Items)
    {
        <li>@ItemTemplate(item)</li>
    }
</ul>

@code {
    [Parameter] public IEnumerable<TItem> Items { get; set; } = default!;
    [Parameter] public RenderFragment<TItem> ItemTemplate { get; set; } = default!;
}
```

Usage:

```
<GenericList Items="employees" Context="emp">
    <ItemTemplate>
        <span>@emp.Name - @emp.Department</span>
    </ItemTemplate>
</GenericList>
```

17.2 Generic Component Constraints

```
@typeparam TEntity where TEntity : IEntity

<MudTable Items="@Items" Hover="true">
    <HeaderContent>
        <MudTh>ID</MudTh>
        <MudTh>Actions</MudTh>
    </HeaderContent>
    <RowTemplate>
        <MudTd>@context.Id</MudTd>  @* IEntity guarantees Id exists *@
        <MudTd>
            <MudIconButton Icon="@Icons.Material.Filled.Edit"
                           OnClick="() => Edit(context)" />
        </MudTd>
    </RowTemplate>
</MudTable>

@code {
    [Parameter] public IEnumerable<TEntity> Items { get; set; } = default!;
    [Parameter] public EventCallback<TEntity> OnEdit { get; set; }

    private async Task Edit(TEntity entity) => await OnEdit.InvokeAsync(entity);
}
```

Appendix A: Quick Reference

Constraint Syntax Summary

Constraint	Meaning
`where T : class`	T must be a reference type
`where T : struct`	T must be a non-nullable value type
`where T : new()`	T must have a parameterless constructor
`where T : BaseClass`	T must inherit from BaseClass
`where T : IInterface`	T must implement IInterface
`where T : notnull`	T must be non-nullable (C# 8+)
`where T : unmanaged`	T must be an unmanaged type (C# 7.3+)
`where T : U`	T must derive from type parameter U

Variance Summary

Keyword	Name	Direction	Safe When
`out`	Covariance	Child → Parent	T is only returned
`in`	Contravariance	Parent → Child	T is only accepted
`(none)`	Invariance	No conversion	T is both input and output

Appendix B: Further Reading

1. **C# Language Specification** — The official reference for generic syntax and semantics
2. **"CLR via C#" by Jeffrey Richter** — Deep dive into how generics work at the runtime level
3. **"C# in Depth" by Jon Skeet** — Excellent coverage of generic evolution across C# versions
4. **Microsoft Docs: Generic Classes** — https://docs.microsoft.com/en-us/dotnet/csharp/programming-guide/generics/
5. **Entity Framework Core Documentation** — Practical examples of generics in ORM contexts

This guide was written as an educational resource for .NET developers seeking to master generics. Understanding generics deeply will improve your ability to write reusable, type-safe, and performant code.